Delivery Route Optimization
The Travelling Salesman Problem with Integer Linear Programming and Genetic Algorithm

Prepared by

Perera R.P.C.T (MS25941012)
De Silva T.H.H.H (MS25941180)

Module: Optimization Methods
Assignment Type: Group Assignment (2 Members)

Table of Contents

Table of Contents 2
1. Introduction and Problem Selection 3
2. Problem and Data Description 3
  2.1 Problem Setup 3
  2.2 Dataset 3
  2.3 Distance Computation 4
  2.4 Constraints 4
3. Mathematical Formulation 5
  3.1 Decision Variables 5
  3.2 Objective Function 5
  3.3 Constraints 5
4. Optimization Methods 6
  4.1 Integer Linear Programming (Exact Method) 6
  4.2 Genetic Algorithm (Metaheuristic) 6
5. Implementation and Repository Structure 7
6. Results and Comparative Analysis 8
  6.1 Solution Summary 8
  6.2 Scalability Analysis 8
7. Discussion 9
  7.1 Key Findings 9
  7.2 Limitations 9
  7.3 Future Work 9
8. Individual Contributions 10
9. References 11

1. Introduction and Problem Selection

This report presents the work we carried out for the Optimization Methods programming assignment. We chose to tackle the **Travelling Salesman Problem (TSP)** and applied it to a realistic delivery-routing scenario. The idea is straightforward: a logistics company based in London needs to deliver goods to 14 other cities across the UK, visiting each one exactly once and then returning to London, all while keeping the total distance as short as possible.

Beyond the academic interest, the TSP has clear practical relevance. Better delivery routes translate directly into lower fuel costs, shorter transit times, and reduced carbon emissions. So this assignment gave us the chance to explore both the theoretical side of optimisation and its real-world applications in logistics and supply-chain management.

2. Problem and Data Description

2.1 Problem Setup

We model the problem on a complete undirected graph G = (V, E), where V = {0, 1, …, 14} is the set of 15 cities and E connects every pair of distinct cities. City 0 (London) is the fixed depot. The goal is to find the shortest Hamiltonian cycle, that is, a round trip that starts at London, visits every other city exactly once, and returns to London.

2.2 Dataset

We used the “Travelling Salesman Problem: Visit 49 UK Cities” dataset from Kaggle, which is available under a CC0 (public domain) licence (Ford, 2021). It provides latitude, longitude, and 2021 census population figures for the 49 largest UK cities. From this set we picked 15 cities that are well spread out geographically, keeping the problem size manageable for the exact solver while still being large enough to be non-trivial.

#	City	Latitude	Longitude	Population
0	London (Depot)	51.5074	–0.1278	8,961,989
1	Birmingham	52.4862	–1.8904	1,153,717
2	Leeds	53.8008	–1.5491	789,194
3	Glasgow	55.8642	–4.2518	635,640
4	Sheffield	53.3811	–1.4701	584,853
5	Manchester	53.4808	–2.2426	552,858
6	Liverpool	53.4084	–2.9916	498,042
7	Edinburgh	55.9533	–3.1883	482,005
8	Bristol	51.4545	–2.5879	467,099
9	Cardiff	51.4816	–3.1791	369,202
10	Leicester	52.6369	–1.1398	357,423

11	Nottingham	52.9548	–1.1581	331,297
12	Newcastle	54.9783	–1.6174	300,196
13	Southampton	50.9097	–1.4044	252,520
14	Plymouth	50.3755	–4.1427	262,172

Table 1. Selected UK cities with geographic coordinates and 2021 census population.

2.3 Distance Computation

We computed pairwise distances using the **Haversine formula**, which gives the great-circle distance between two points on a sphere based on their latitude and longitude. It is not a perfect reflection of actual road distances, but it is a well-established approximation that works well for benchmarking purposes.

2.4 Constraints

The constraints are simple: (i) every city must be visited exactly once; (ii) the tour must start and finish at London; and (iii) all 15 cities must be included.

3. Mathematical Formulation

We formulated the TSP as an Integer Linear Programme (ILP) using the Miller–Tucker–Zemlin (MTZ) subtour elimination approach (Miller et al., 1960). This is a classic formulation that is well suited to our problem size.

3.1 Decision Variables

Two sets of decision variables are used:

Binary routing variables: $x_{ij} \in \{0, 1\}$ for all $i, j \in V$ with $i \neq j$. If the tour goes directly from city i to city j, then $x_{ij} = 1$; otherwise it is 0.

Ordering variables: $u_i \in \{1, 2, \ldots, n - 1\}$ for each city $i \neq 0$. These encode the position of each city in the tour and are the key to preventing subtours.

3.2 Objective Function

The objective is to minimise the total tour distance:

$$\textit{Minimise} \quad Z = \Sigma_i \Sigma_j \; d_{ij} \cdot x_{ij} \qquad \textit{for all } i, j \in V, i \neq j$$

where d_{ij} is the Haversine distance between cities i and j.

3.3 Constraints

Assignment constraints: Each city must have exactly one arc coming in and one going out. Formally, for every city i, the sum of x_{ij} over all $j \neq i$ equals 1, and similarly the sum of x_{ij} over all $i \neq j$ equals 1.

MTZ subtour elimination: For all $i, j \in V \setminus \{0\}$ with $i \neq j$, we require $u_i - u_j + n \cdot x_{ij} \leq n - 1$. The effect is that if $x_{ij} = 1$, then u_j must be at least $u_i + 1$, which prevents disconnected loops. The whole formulation gives us $O(n^2)$ binary variables and $O(n^2)$ constraints.

4. Optimization Methods

As required by the assignment brief, we implemented two different approaches: one exact method and one metaheuristic. This lets us compare the trade-offs between guaranteed optimality and computational efficiency.

4.1 Integer Linear Programming (Exact Method)

Perera R.P.C.T (MS25941012) implemented the ILP formulation from Section 3 using the PuLP modelling library with the COIN-OR Branch and Cut (CBC) solver. Being an exact method, ILP guarantees that the solution it finds is provably optimal. Under the hood, CBC uses branch and bound: it solves the LP relaxation first, then branches on fractional variables and prunes subproblems whose lower bounds exceed the best solution found so far.

The clear advantage is that you get the best possible answer, and it is fully deterministic; run it again and you get the same result. The downside, as you would expect with an NP-hard problem, is that computation time grows exponentially as the number of cities increases.

4.2 Genetic Algorithm (Metaheuristic)

De Silva T.H.H.H (MS25941180) implemented the Genetic Algorithm (GA) using the DEAP framework (Distributed Evolutionary Algorithms in Python). GAs are population-based metaheuristics inspired by natural selection (Holland, 1975; Goldberg, 1989). A set of candidate tours evolves over successive generations through selection, crossover, and mutation.

The parameter configuration we settled on after some experimentation is shown below:

Parameter	Value	Justification
Population Size	200	Enough diversity for 15 cities
Generations	500	Allows convergence without excessive runtime
Selection	Tournament (k = 3)	Good balance of pressure and diversity
Crossover	Ordered Crossover (OX)	Preserves valid permutations
Mutation Rate	0.2 (swap, p = 0.05)	Adds variation without being too disruptive

Table 2. Genetic Algorithm parameter configuration.

The GA runs in polynomial time, specifically $O(G \cdot P \cdot n)$, where G is generations, P is population size, and n is the number of cities. It cannot guarantee the global optimum, but it scales much better than exact methods for larger instances.

5. Implementation and Repository Structure

Everything was written in **Python 3** using **Jupyter Notebooks** as the main development environment. The key libraries are PuLP for the ILP model, DEAP for the genetic algorithm, NumPy and Pandas for data handling, and Matplotlib for plotting.

The project lives in a GitHub repository with a proper commit history showing contributions from both of us. The structure is as follows:

Directory / File	Description
notebooks/	Jupyter notebooks for both methods
notebooks/chamod_exact_method.ipynb	ILP implementation and analysis
notebooks/hirantha_genetic_algorithm.ipynb	Genetic Algorithm implementation
data/UK_Cities.csv	Source dataset with city coordinates
data/ilp_results.json	ILP solver output and metrics
data/ga_results.json	GA execution output and metrics
report/report.md	Report source in Markdown
code_appendix.py	Consolidated code listing
members.txt	Group member information
submission.txt	Dataset, GitHub, and YouTube links

Table 3. Repository structure and file descriptions.

6. Results and Comparative Analysis

6.1 Solution Summary

We ran both methods on the full 15-city instance. The ILP solver returned a provably optimal tour, while the GA produced a near-optimal solution. The exact distance values, solve times, and optimality gap are generated from the notebook runs and stored in the accompanying JSON files. A qualitative comparison is summarised below.

Metric	ILP (Exact)	GA (Metaheuristic)
Solution Quality	Optimal (proven)	Near-optimal
Optimality Guarantee	Yes	No
Deterministic	Yes	No (stochastic)
Scalability	Exponential	Polynomial
Runtime Growth	Exponential with n	Approx. linear with n

Table 4. Comparative summary of ILP and GA performance.

6.2 Scalability Analysis

To get a clearer picture of how the two methods scale, we tested them on subsets of increasing size: 5, 8, 10, 12, and 15 cities. As expected, the ILP solver's runtime shot up exponentially as we added more cities, which is consistent with the NP-hard nature of the problem. The GA, on the other hand, showed roughly linear growth in execution time, confirming that it is much more practical for larger instances.

7. Discussion

7.1 Key Findings

The main takeaway from this assignment is really about the trade-off between solution quality and computational scalability. The ILP gives you the proven best answer, but at a cost that becomes impractical as the problem grows. The GA gets you a very good answer, often within a few percent of optimal, in a fraction of the time.

Looking at the GA's convergence behaviour, most of the improvement happens in the first 100 or so generations, after which progress flattens out. Our parameter sensitivity analysis also showed that population size has a bigger impact on solution quality than the number of generations, which makes sense because maintaining diversity in the population is what keeps the search from getting stuck in local optima.

7.2 Limitations

There are a few limitations worth acknowledging. First, the Haversine distance is a straight-line approximation that does not account for actual road networks, traffic, or terrain. Second, we assumed a single vehicle with no capacity limits, which is a simplification compared to real delivery operations. Third, the problem is treated as static, whereas in practice you would have time windows, fluctuating demand, and live traffic data to deal with.

7.3 Future Work

There are several directions we would like to explore if we had more time. A memetic algorithm that combines the GA with a 2-opt local search could potentially get the best of both worlds: global exploration with local refinement. Testing on the full 49-city dataset would be a more serious scalability test. Using real road distances from the Google Maps or OpenStreetMap API would make the results more practically meaningful. We could also generalise the problem to the Vehicle Routing Problem (VRP) with multiple vehicles and capacity constraints. Finally, it would be interesting to implement Ant Colony Optimisation as another metaheuristic to compare against the GA.

8. Individual Contributions

We split the work fairly evenly. Each of us took the lead on one optimisation method while pitching in on shared tasks like problem selection and the report. The table below shows the breakdown.

Task	Perera R.P.C.T	De Silva T.H.H.H
Problem Selection	Joint	Joint
Dataset Selection	Lead	Reviewed
Mathematical Formulation	Lead	Reviewed
ILP Implementation	Sole Responsibility	-
GA Implementation	-	Sole Responsibility
Parameter Tuning (GA)	-	Sole Responsibility
Route Map & Heatmap	Sole Responsibility	-
Convergence & Comparison Plots	-	Sole Responsibility
Comparative Analysis	Contributed	Contributed
Report (Sections 1–3, 4.1)	Reviewed	Reviewed
Report (Sections 4.2, 6–7)	Reviewed	Reviewed
Video Presentation	First Half	Second Half

Table 5. Individual contribution matrix.

9. References

[1] Applegate, D.L., Bixby, R.E., Chvátal, V. and Cook, W.J. (2006) *The Traveling Salesman Problem: A Computational Study*. Princeton University Press.

[2] Ford, P. (2021) “Travelling Salesman Problem: Visit 49 UK Cities” [Dataset]. Kaggle. Available at: https://www.kaggle.com/datasets/patricklford/travelling-salesman-problem (Accessed: April 2026).

[3] Goldberg, D.E. (1989) *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley.

[4] Holland, J.H. (1975) *Adaptation in Natural and Artificial Systems*. University of Michigan Press.

[5] Miller, C.E., Tucker, A.W. and Zemlin, R.A. (1960) “Integer Programming Formulation of Traveling Salesman Problems”, *Journal of the ACM*, 7(4), pp. 326–329.

[6] COIN-OR PuLP (2024) *PuLP: A Linear Programming Toolkit for Python*. Available at: https://coin-or.github.io/pulp/ (Accessed: April 2026).

[7] DEAP (2024) *Distributed Evolutionary Algorithms in Python*. Available at: https://deap.readthedocs.io/ (Accessed: April 2026).